

五级流水RISC-V处理器设计与实现

竞业达-RISCV CPU设计赛

未命名队 /CICC1001127

汇报人：王娅蓁

2026 年 7 月

JYD 竞业达

目录

CONTENT

01 项目概述

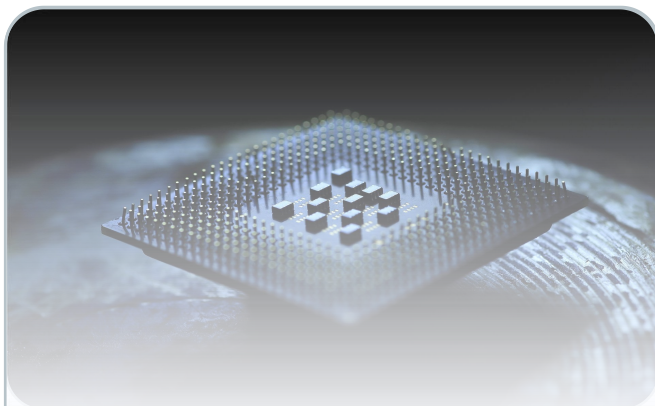
02 整体架构图

03 模块设计

亮点与关键技术
各模块详解

04 结果展示

仿真结果
验证结果
优化过程



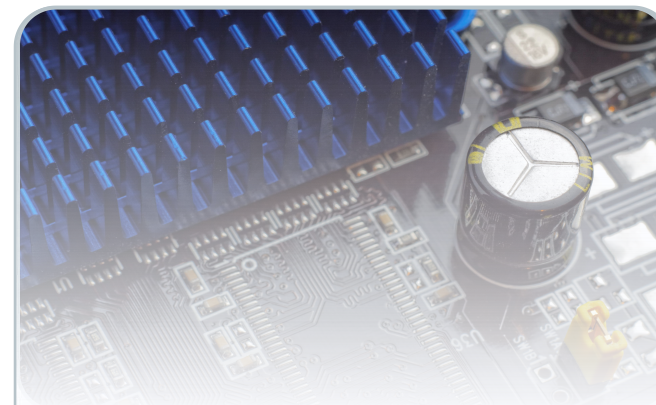
项目目标

- 实现基于RISC-V指令集的多级流水线架构CPU设计。
- 目前实现了**五级流水线**，并实现了**分支预测**模块。

AMD
Vivado

平台工具

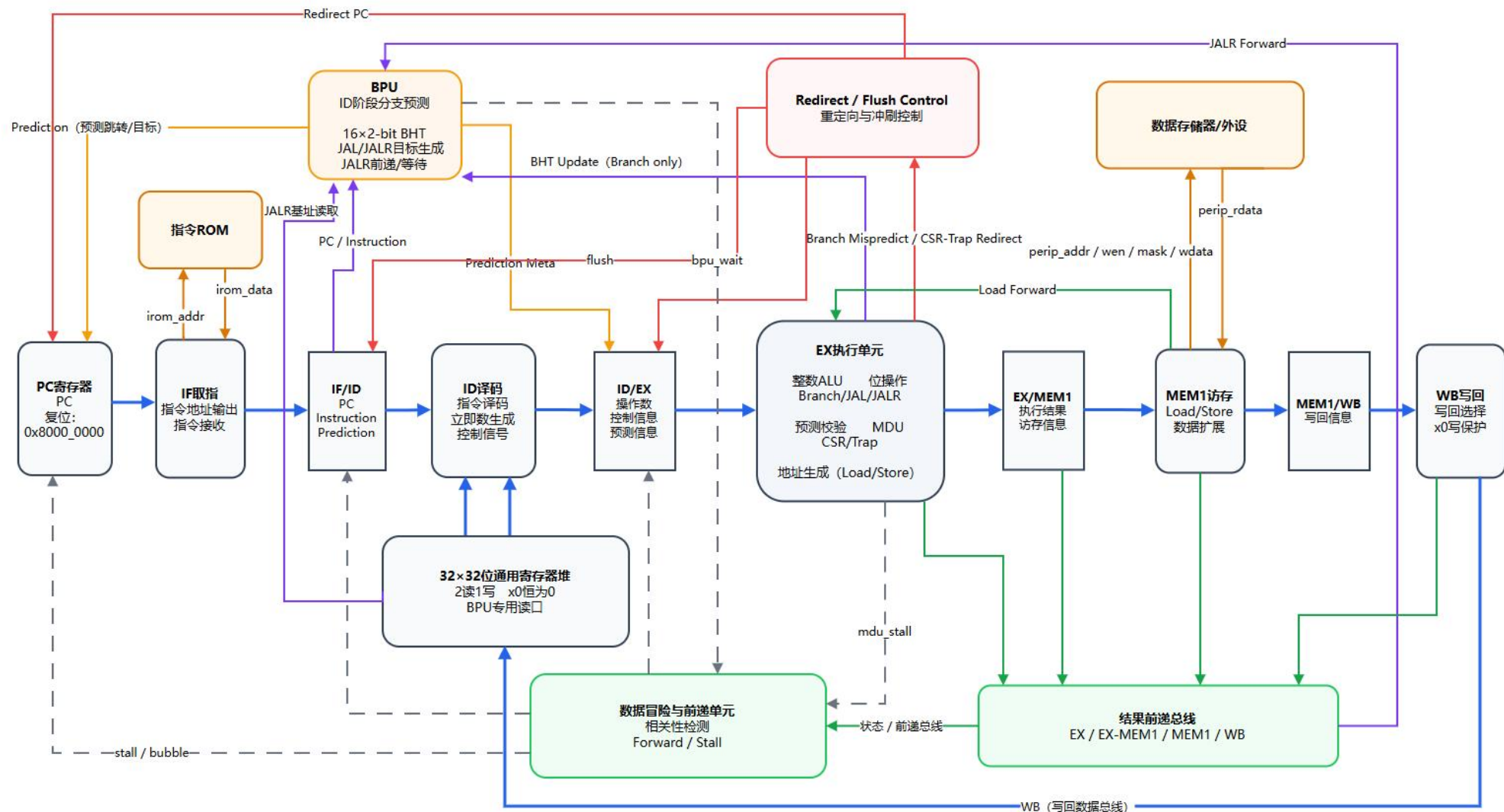
- **设计平台**
Vivado2023.2
- **仿真工具**
Vivado Simulation (2023.2)
- **验证平台**
官方孪生平台
Xilinx K-7 (FPGA型号)



实践意义

- 通过实践熟悉**RISC-V指令集**，学会设计流水线、数据通路，提升硬件设计能力。

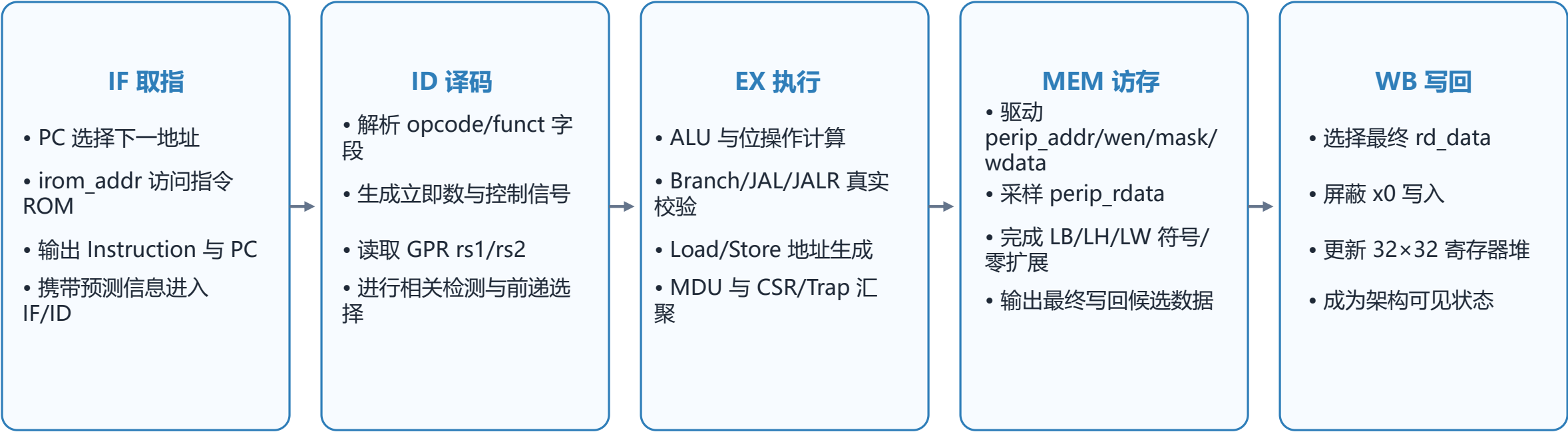
02.竞业达RISCV CPU设计竞赛--整体架构图



读图方式：蓝色为主数据通路，橙色为预测通路，紫色为BPU信号，红色为真实重定向/冲刷，绿色为前递，灰色虚线为停顿控制。

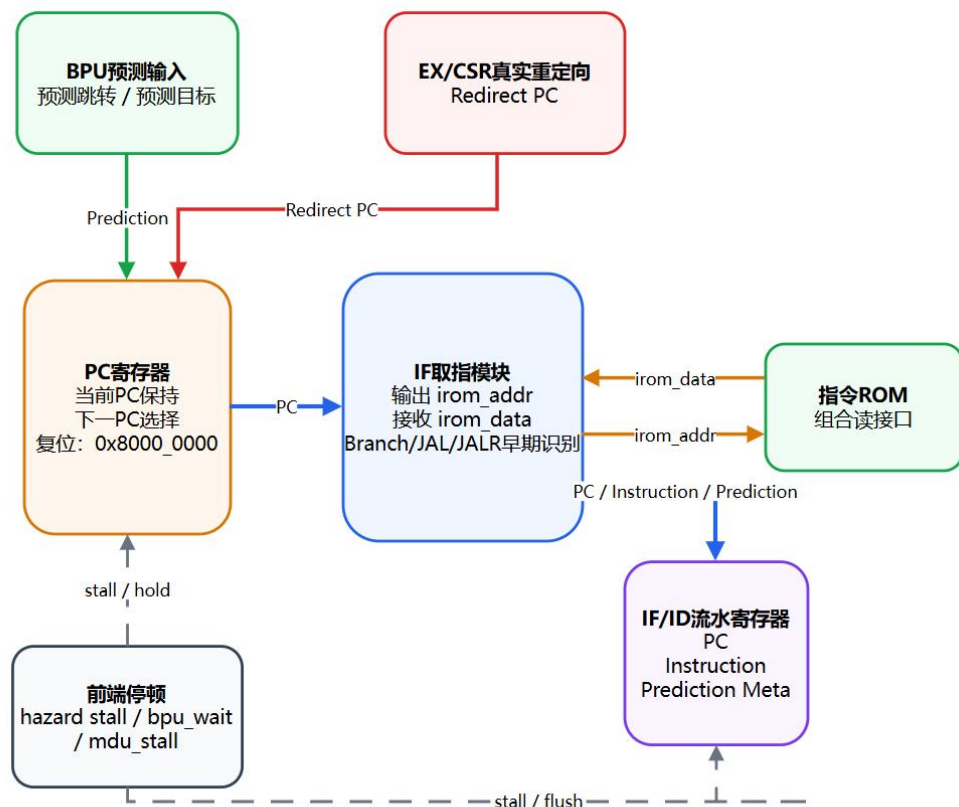
03.竞业达RISCV CPU设计竞赛--模块功能设计

一条指令在流水线中不是只携带“数据”，还要同步携带 PC、控制字段、目的寄存器、访存属性、系统操作和预测信息。



控制动作	发生条件	设计语义
保持 Stall	BPU等待、Load相关、MDU busy	当前内容仍正确，只是暂时不能推进
气泡 Bubble	Load-Use 消费者进入 EX 前	向后级写入无副作用 NOP，让老指令继续
冲刷 Flush	误预测、CSR/Trap 重定向	删除错误路径年轻指令，防止写回/Store/CSR副作用

03.竞业达RISCV CPU设计竞赛--IF模块功能设计



● 功能：

完成**取指**操作、**跳转**、**简单译码**传递信号给分支预测单元

● 关键子模块：

从PC_reg中取出指令地址完成**取指**操作

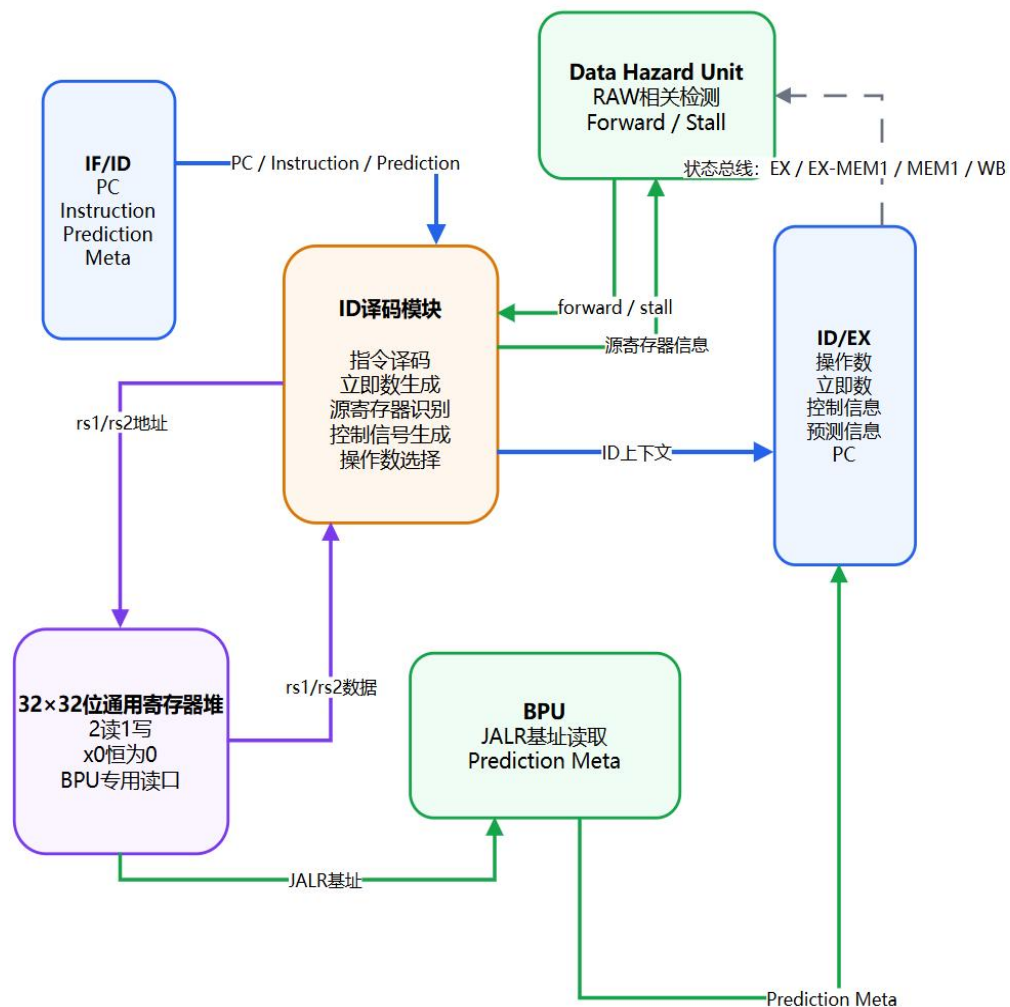
对**分支跳转指令简单译码**，传递信号给分支预测单元 (bpu)

PC_reg根据EX单元、分支预测单元以及冒险单元提供的信号进行更新

● 信号流：

PC → ROM → IF/ID流水线寄存器

PC → IF → bpu分支预测模块 → PC更新

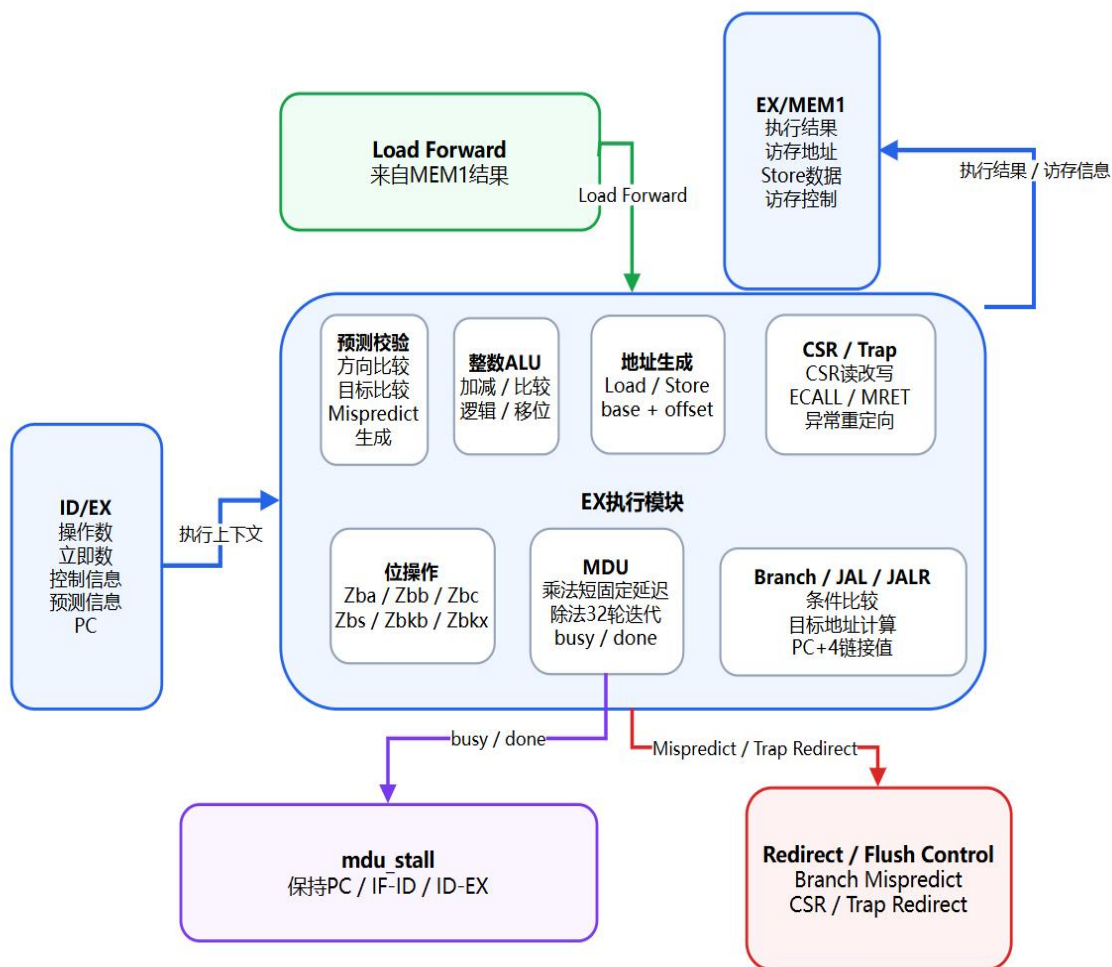


• 功能：

解析指令并生成控制信号与操作数，完成**读取寄存器**，传递信号给**流水线寄存器**以及**数据冒险单元**

• 核心机制：

ID 模块完成完整**译码**、**立即数生成**、**寄存器读取与控制字段生成**，并把执行所需的“指令上下文”**整体**送入ID/EX，而不是只送部分数据。通用寄存器堆为32×32位，2读1写，x0恒为0，同时支持**面向BPU**的专用读口。数据冒险检测单元在ID阶段比较源寄存器与EX、MEM、WB的目的寄存器，决定是否**前递或停顿**。



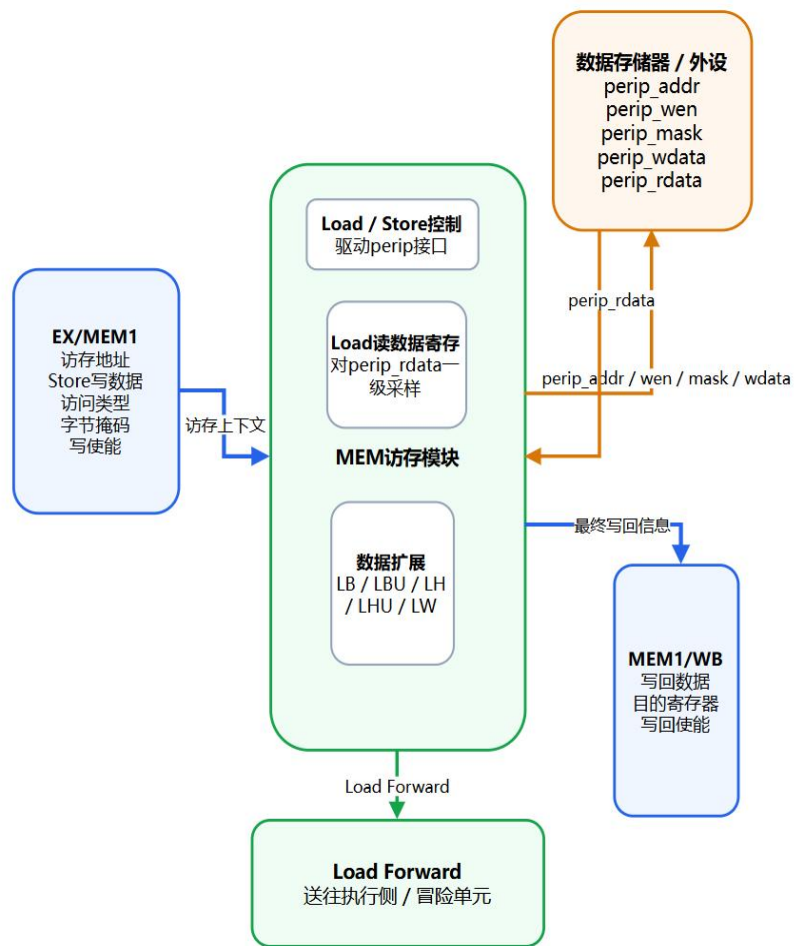
• 语义确认点:

普通指令由**ALU**产生结果，分支在此比较并**计算真实目标**，Load/Store在此**生成**地址，MDU在此**启动**，CSR/异常在此**决定是否重定向**。

• 核心机制:

分支、JAL与JALR的**真实后继**都在EX阶段确认；若**预测错误**，则产生 `real_jump_en / real_jump_addr`，并由ctrl**清除**年轻级错误路径。内部还集成了RV32M**乘除法单元MDU**。**乘法**在**短固定延迟**内完成，**除法**采用**32轮**迭代；**busy**期间流水线**保持**，**done**后再**开放写回**。**CSR指令和异常处理**也在EX相关路径完成。

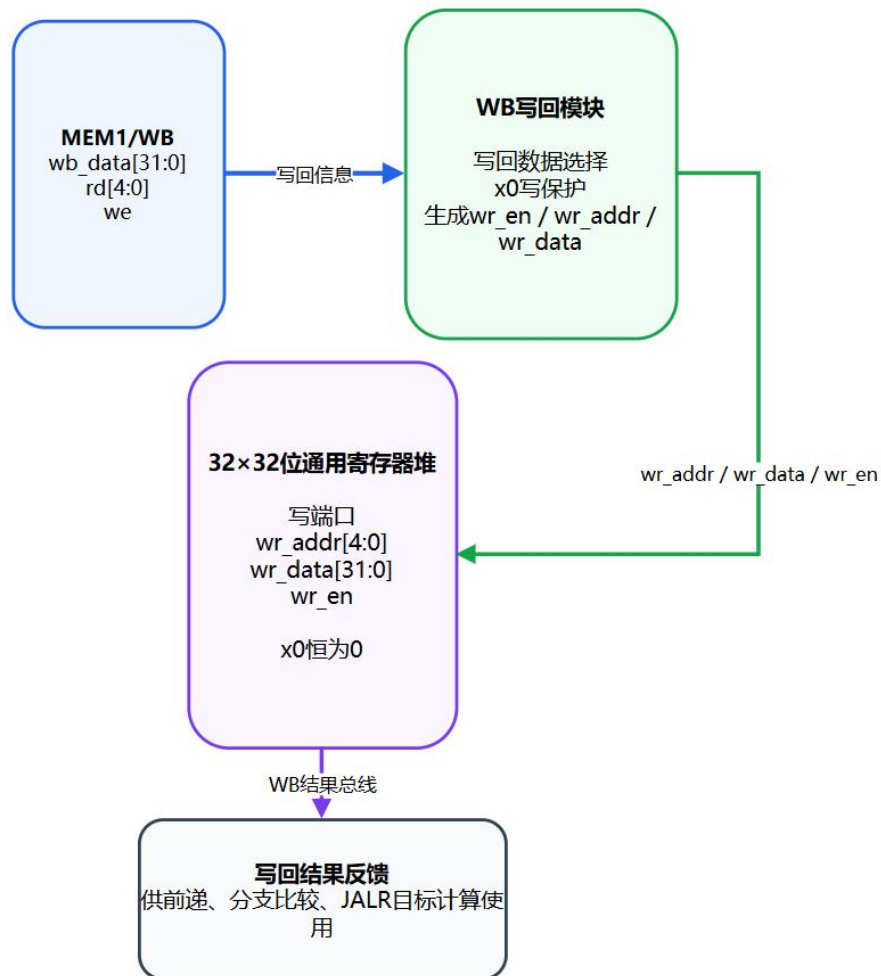
03.竞业达RISCV CPU设计竞赛--MEM模块功能设计



• 功能:

MEM负责把**外部perip接口**和流水线**内部写回链**衔接起来。

- EX/MEM将**地址、写数据、访存宽度和控制信息**送入MEM; Store在MEM通过perip_wen / perip_mask / perip_wdata对外产生**写副作用**。Load的关键特点是结果不在EX形成,而是在MEM中对perip_rdata进行**一级时序采样**,并在**后一拍**完成LB/LBU、LH/LHU、LW的符号或零扩展。平台数据侧采用**固定延迟**perip接口,没有valid/ready握手,因此当前设计通过**MEM寄存读数据**来保证时序与正确性。



- **功能:**

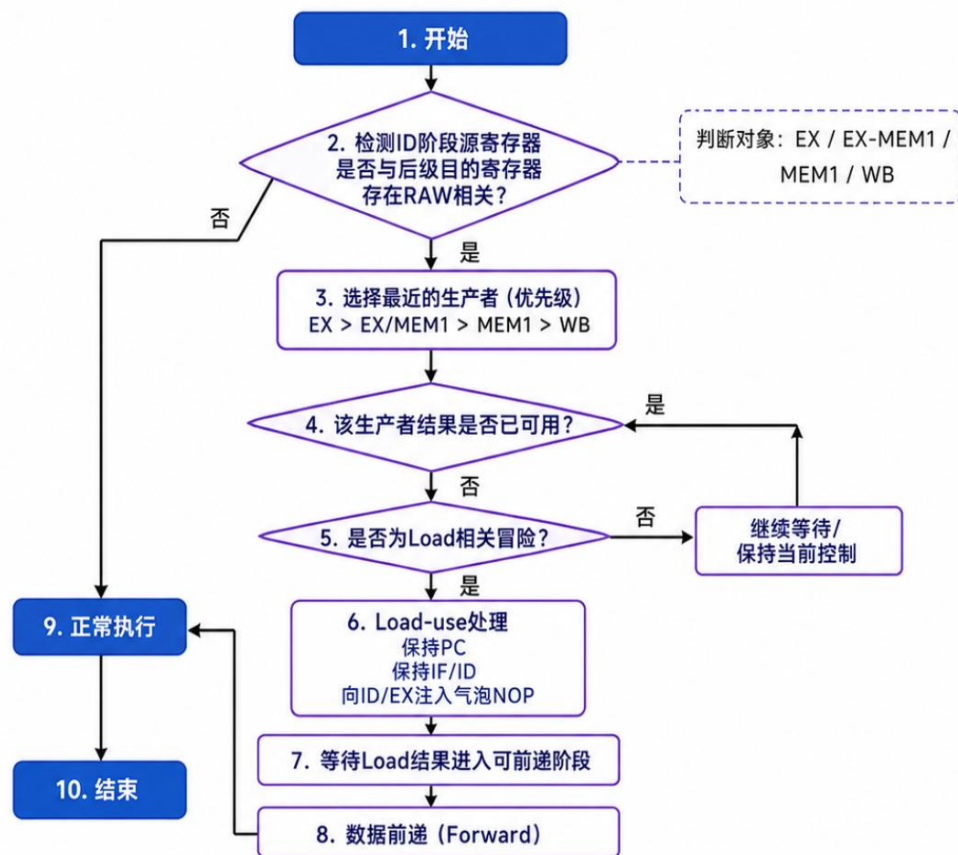
将数据写回寄存器堆。

- **基本逻辑:**

时序写，**下一个时钟上升沿**到来时，数据写入寄存器堆。

检查写回的寄存器是否是x0，若是x0寄存器，则不写入新数据。

通用寄存器的架构状态**统一**在 WB 更新，这样便于围绕单一写入口分析前递、异常恢复和调试波形。



数据冒险单元**集中检测ID阶段**源寄存器与EX、EX/MEM、MEM、WB阶段目的寄存器之间的**RAW**相关。

对于结果已就绪的相关内容，按照距离消费者**最近**的生产者优先进行**多级前递**；

对于尚未完成的**Load**相关，保持PC和IF/ID，并向ID/EX**注入气泡**，直至寄存后的Load数据**进入可前递**阶段。

Branch和**JALR**还具有**针对前端**控制流操作数的专用等待机制。

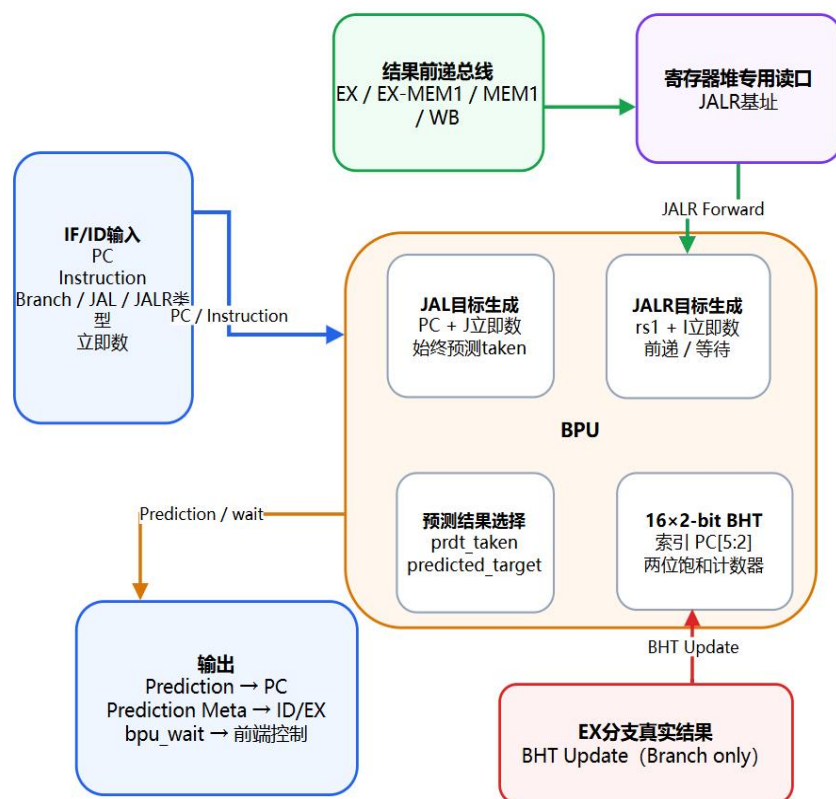
功能：

预测跳转和分支并计算目标地址。

核心机制：

针对**B型**条件分支，采用**16项两位饱和计数器BHT**进行动态方向预测。JAL与JALR指令直接进行跳转。

由于JALR指令目标地址计算**依赖rs1**，可能与**前序尚未写回**的结果产生数据相关，因此BPU增设了面向JALR的相关**检测、数据前递与等待逻辑**；当数据尚未就绪时，前端保持等待，待结果可用后再继续预测。



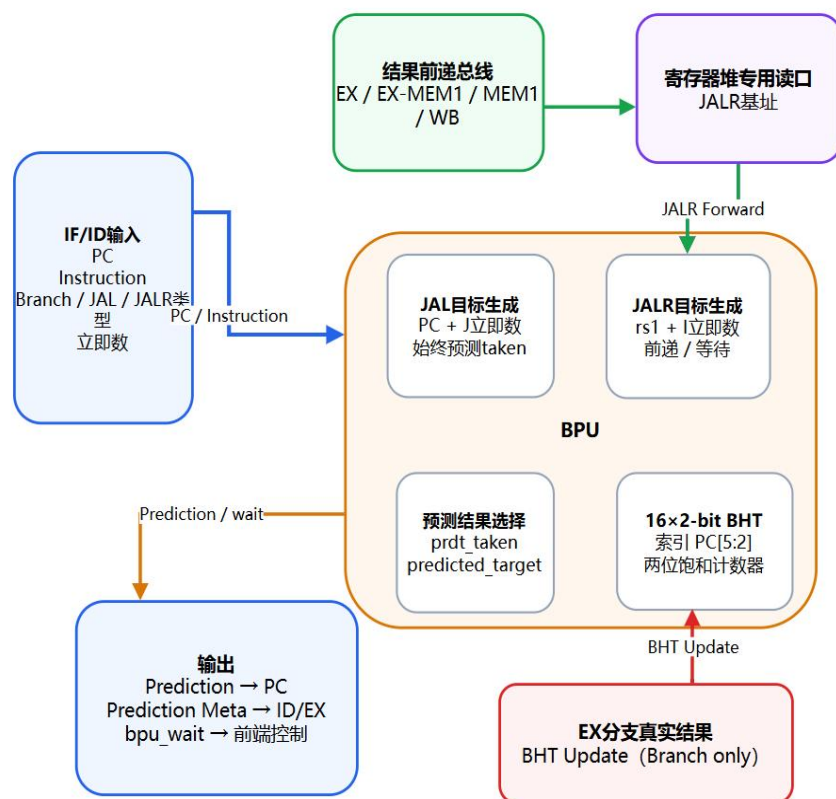
功能：

预测跳转和分支并计算目标地址。

核心机制：

针对**B型**条件分支，采用**16项两位饱和计数器BHT**进行动态方向预测。JAL与JALR指令直接进行跳转。

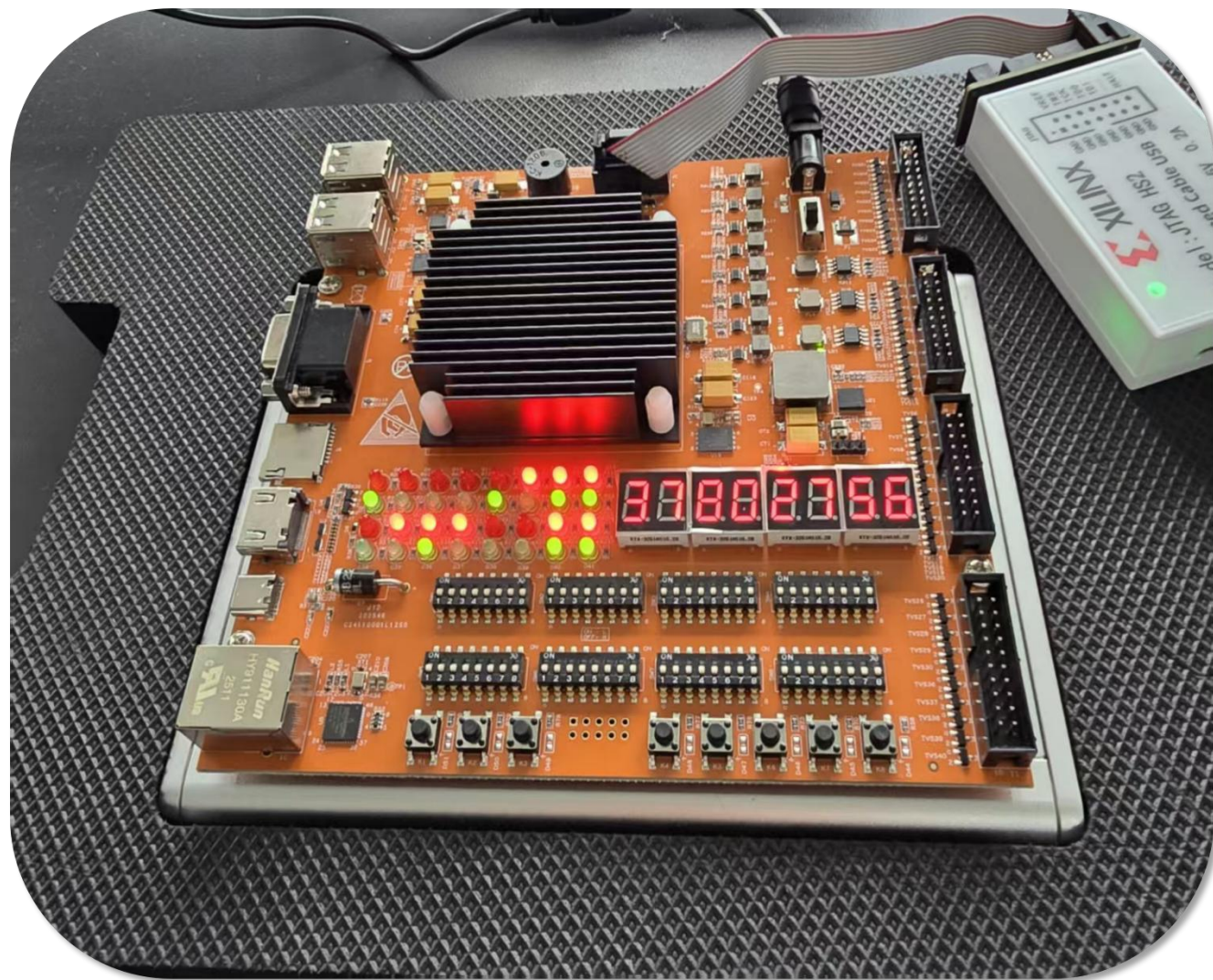
由于JALR指令目标地址计算**依赖rs1**，可能与**前序尚未写回**的结果产生数据相关，因此BPU增设了面向JALR的相关**检测、数据前递与等待逻辑**；当数据尚未就绪时，前端保持等待，待结果可用后再继续预测。



仿真结果展示

经过对不同类型指令的仿真分析，指令的执行顺序与流水线的工作均符合设计预期，能够通过官方给出的测试文件。具体分析过程在仿真报告中有所体现。





测试结果

对官方提供的WithMext
测试结果如图所示

竞业达RISCV CPU设计竞赛--结论与总结



1. 指令集覆盖

RV32I 基础整数; RV32M 乘除法; Zicsr 六类 CSR; Zba/Zbb/Zbc/Zbs/Zbkb/Zbkx 位操作子集。

2. 流水线组织

IF、ID、EX、MEM1、WB 五级顺序流水; 级间寄存器同步保存完整指令上下文。

3. 数据相关处理

EX/EX-MEM1/MEM1/WB 多级前递; Load-Use 保持前端并向 ID/EX 注入气泡。

提交版 RTL 功能闭环

4. 控制流处理

16×2-bit BHT 动态方向预测; JAL/JALR 预测跳转; EX 校验真实方向并冲刷错误路径。

5. 执行与系统单元

EX 集成 ALU、位操作、地址生成、MDU、CSR/Trap; MDU 支持乘法和 32 轮除法迭代。

6. 平台接口与写回

组合 IROM 取指; perip 数据侧接口; MEM1 寄存 Load 数据并完成扩展; WB 屏蔽 x0 写入。

覆盖场景: 算术逻辑 | Load/Store | 分支/JAL/JALR | 乘除法 | CSR读改写 | 异常进入/返回 | 板级固定延迟接口

问题与优化方向

当前局限:

- BHT无tag, 仅方向预测, 易别名冲突
- 固定延迟访存接口, 无Cache, 性能受限
- MDU阻塞式设计, 单发射流水IPC上限低

优化方向:

- 增加BTB/RAS, 升级分支预测器
- 引入I/D Cache + AXI总线接口
- MDU流水化 / 双发射 / 中断支持

Q & A

谢谢大家

恳请各位老师批评与指正！